

Министерство образования и науки Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
02.03.01 Математика и компьютерные науки

Отчёт по дисциплине
Вычислительная математика
Лабораторная работа №5
«Дискретное преобразование Фурье»
Вариант 18

Выполнил:
студент группы 5130201/40003
Джабраилов А.В.
Подпись: _____

Принял:
К. физ.-мат. наук, доц., доц.
Пак В.Г.
Подпись: _____
«_____» _____ 2026 г.

Санкт-Петербург, 2026

1 Задание

1. Применяя дискретное преобразование Фурье для данного набора сигналов, получить вектор амплитуд, затем по нему обратным преобразованием получить исходный сигнал.
2. Применяя быстрое преобразование Фурье с прореживанием по времени по основанию 2 для данного набора сигналов, получить векторы амплитуд по чётным и нечётным отсчётам, вектор амплитуд по всем отсчётам; затем по нему обратным преобразованием получить векторы обратных преобразований по чётным и нечётным отсчётам и исходный сигнал.

2 Цель работы

Освоить алгоритмы дискретного и быстрого преобразований Фурье, реализовать прямое и обратное преобразования для заданного дискретного сигнала и проверить восстановление исходных отсчётов.

3 Ход работы

Исходные данные варианта 18 содержат $n = 16$ временных отсчётов.

Таблица 1: Таблица исходных отсчётов.

j	0	1	2	3	4	5	6	7
y_j	-0,78	-1,22	-1,34	-1,39	-1,42	-1,43	-1,42	-1,41

j	8	9	10	11	12	13	14	15
y_j	-1,37	-1,30	-1,10	-0,10	1,12	1,25	1,33	1,36

Алгоритм вычислений:

1. Вычислить вектор комплексных амплитуд F_k по формуле прямого дискретного преобразования Фурье.
2. По найденному вектору F_k выполнить обратное дискретное преобразование Фурье и сравнить результат с исходным сигналом.
3. Разделить исходный вектор на отсчёты с чётными и нечётными индексами.
4. Для полученных подвекторов вычислить векторы амплитуд F_k^0 и F_k^1 .
5. По формулам быстрого преобразования Фурье получить общий вектор амплитуд F_k .
6. Выполнить обратные преобразования для F_k^0 и F_k^1 , затем восстановить исходный сигнал.

Для реализации использован язык *Python*. В программе все преобразования вычисляются явно по формулам методических указаний, без использования готовой функции БПФ.

3.1 Дискретное преобразование Фурье

Прямое дискретное преобразование Фурье определяется формулой:

$$F_k = \sum_{j=0}^{n-1} y_j \exp \left(-\frac{2\pi i k}{n} j \right), \quad k = 0, 1, \dots, n-1.$$

Обратное преобразование определяется формулой:

$$y_j = \frac{1}{n} \sum_{k=0}^{n-1} F_k \exp \left(\frac{2\pi i k}{n} j \right), \quad j = 0, 1, \dots, n-1.$$

Подставим $n = 16$ и значения исходных отсчётов в формулу прямого ДПФ:

$$\begin{aligned} F_k = & -0,78 \exp \left(-\frac{2\pi i k}{16} \cdot 0 \right) - 1,22 \exp \left(-\frac{2\pi i k}{16} \cdot 1 \right) - 1,34 \exp \left(-\frac{2\pi i k}{16} \cdot 2 \right) \\ & - 1,39 \exp \left(-\frac{2\pi i k}{16} \cdot 3 \right) - 1,42 \exp \left(-\frac{2\pi i k}{16} \cdot 4 \right) - 1,43 \exp \left(-\frac{2\pi i k}{16} \cdot 5 \right) \\ & - 1,42 \exp \left(-\frac{2\pi i k}{16} \cdot 6 \right) - 1,41 \exp \left(-\frac{2\pi i k}{16} \cdot 7 \right) - 1,37 \exp \left(-\frac{2\pi i k}{16} \cdot 8 \right) \\ & - 1,30 \exp \left(-\frac{2\pi i k}{16} \cdot 9 \right) - 1,10 \exp \left(-\frac{2\pi i k}{16} \cdot 10 \right) - 0,10 \exp \left(-\frac{2\pi i k}{16} \cdot 11 \right) \\ & + 1,12 \exp \left(-\frac{2\pi i k}{16} \cdot 12 \right) + 1,25 \exp \left(-\frac{2\pi i k}{16} \cdot 13 \right) \\ & + 1,33 \exp \left(-\frac{2\pi i k}{16} \cdot 14 \right) + 1,36 \exp \left(-\frac{2\pi i k}{16} \cdot 15 \right). \end{aligned}$$

Например, для первых двух амплитуд:

$$\begin{aligned} F_0 = & -0,78 - 1,22 - 1,34 - 1,39 - 1,42 - 1,43 - 1,42 - 1,41 \\ & - 1,37 - 1,30 - 1,10 - 0,10 + 1,12 + 1,25 + 1,33 + 1,36 = -9,220000, \\ F_1 \approx & 5,529825 + 9,351469i. \end{aligned}$$

Подставим найденные амплитуды в формулу обратного ДПФ:

$$\begin{aligned} \hat{y}_j = & \frac{1}{16} \left(-9,220000 \exp \left(\frac{2\pi i \cdot 0}{16} j \right) + (5,529825 + 9,351469i) \exp \left(\frac{2\pi i \cdot 1}{16} j \right) \right. \\ & + (-2,486396 + 5,022864i) \exp \left(\frac{2\pi i \cdot 2}{16} j \right) + (-1,378383 + 0,540232i) \exp \left(\frac{2\pi i \cdot 3}{16} j \right) \\ & + (0,080000 + 1,160000i) \exp \left(\frac{2\pi i \cdot 4}{16} j \right) + (-0,991293 + 1,391733i) \exp \left(\frac{2\pi i \cdot 5}{16} j \right) \\ & + (-1,213604 + 0,322864i) \exp \left(\frac{2\pi i \cdot 6}{16} j \right) + (-0,800149 + 0,042971i) \exp \left(\frac{2\pi i \cdot 7}{16} j \right) \\ & - 0,740000 \exp \left(\frac{2\pi i \cdot 8}{16} j \right) + (-0,800149 - 0,042971i) \exp \left(\frac{2\pi i \cdot 9}{16} j \right) \\ & + (-1,213604 - 0,322864i) \exp \left(\frac{2\pi i \cdot 10}{16} j \right) + (-0,991293 - 1,391733i) \exp \left(\frac{2\pi i \cdot 11}{16} j \right) \\ & + (0,080000 - 1,160000i) \exp \left(\frac{2\pi i \cdot 12}{16} j \right) + (-1,378383 - 0,540232i) \exp \left(\frac{2\pi i \cdot 13}{16} j \right) \\ & \left. + (-2,486396 - 5,022864i) \exp \left(\frac{2\pi i \cdot 14}{16} j \right) + (5,529825 - 9,351469i) \exp \left(\frac{2\pi i \cdot 15}{16} j \right) \right). \end{aligned}$$

3.2 Быстрое преобразование Фурье

Вектор отсчётов разделяется на два подвектора половинной длины:

$$y_j^0 = y_{2j}, \quad y_j^1 = y_{2j+1}, \quad j = 0, 1, \dots, \frac{n}{2} - 1.$$

Для варианта 18:

$$\bar{y}^0 = (-0,78, -1,34, -1,42, -1,42, -1,37, -1,10, 1,12, 1,33),$$

$$\bar{y}^1 = (-1,22, -1,39, -1,43, -1,41, -1,30, -0,10, 1,25, 1,36).$$

Введём обозначение из методических указаний:

$$W_n^{j \cdot k} = \exp \left(-\frac{2\pi i k}{n} j \right).$$

Тогда для подвекторов:

$$F_k^0 = \sum_{j=0}^{\frac{n}{2}-1} y_{2j} W_{\frac{n}{2}}^{j \cdot k}, \quad F_k^1 = \sum_{j=0}^{\frac{n}{2}-1} y_{2j+1} W_{\frac{n}{2}}^{j \cdot k}.$$

Подставим $n = 16$, то есть для подвекторов используется $W_8^{j \cdot k}$:

$$\begin{aligned} F_k^0 &= -0,78 \exp \left(-\frac{2\pi i k}{8} \cdot 0 \right) - 1,34 \exp \left(-\frac{2\pi i k}{8} \cdot 1 \right) - 1,42 \exp \left(-\frac{2\pi i k}{8} \cdot 2 \right) \\ &\quad - 1,42 \exp \left(-\frac{2\pi i k}{8} \cdot 3 \right) - 1,37 \exp \left(-\frac{2\pi i k}{8} \cdot 4 \right) - 1,10 \exp \left(-\frac{2\pi i k}{8} \cdot 5 \right) \\ &\quad + 1,12 \exp \left(-\frac{2\pi i k}{8} \cdot 6 \right) + 1,33 \exp \left(-\frac{2\pi i k}{8} \cdot 7 \right), \\ F_k^1 &= -1,22 \exp \left(-\frac{2\pi i k}{8} \cdot 0 \right) - 1,39 \exp \left(-\frac{2\pi i k}{8} \cdot 1 \right) - 1,43 \exp \left(-\frac{2\pi i k}{8} \cdot 2 \right) \\ &\quad - 1,41 \exp \left(-\frac{2\pi i k}{8} \cdot 3 \right) - 1,30 \exp \left(-\frac{2\pi i k}{8} \cdot 4 \right) - 0,10 \exp \left(-\frac{2\pi i k}{8} \cdot 5 \right) \\ &\quad + 1,25 \exp \left(-\frac{2\pi i k}{8} \cdot 6 \right) + 1,36 \exp \left(-\frac{2\pi i k}{8} \cdot 7 \right). \end{aligned}$$

По формулам БПФ:

$$\begin{cases} F_k = F_k^0 + W_n^k F_k^1, \\ F_{k+\frac{n}{2}} = F_k^0 - W_n^k F_k^1, \end{cases} \quad k = 0, \dots, \frac{n}{2} - 1.$$

Например, для $k = 1$:

$$\begin{aligned} F_1 &= (2,364838 + 4,654249i) + \exp \left(-\frac{2\pi i}{16} \right) (1,126518 + 5,550854i) \\ &= 5,529825 + 9,351469i, \\ F_9 &= (2,364838 + 4,654249i) - \exp \left(-\frac{2\pi i}{16} \right) (1,126518 + 5,550854i) \\ &= -0,800149 - 0,042971i. \end{aligned}$$

Векторы обратных преобразований по чётным и нечётным отсчётам вычисляются по формуле обратного ДПФ для векторов длины 8:

$$\hat{y}_j^0 = \frac{1}{8} \sum_{k=0}^7 F_k^0 \exp\left(\frac{2\pi i k}{8} j\right), \quad \hat{y}_j^1 = \frac{1}{8} \sum_{k=0}^7 F_k^1 \exp\left(\frac{2\pi i k}{8} j\right).$$

Исходный сигнал восстанавливается обратным объединением подвекторов:

$$\hat{y}_{2j} = \hat{y}_j^0, \quad \hat{y}_{2j+1} = \hat{y}_j^1.$$

4 Результаты

4.1 Дискретное преобразование Фурье

Вектор комплексных амплитуд, полученный по формуле прямого ДПФ:

Таблица 2: Вектор амплитуд $F(k)$.

k	$F(k)$	k	$F(k)$
0	$-9,220000$	8	$-0,740000$
1	$5,529825 + 9,351469i$	9	$-0,800149 - 0,042971i$
2	$-2,486396 + 5,022864i$	10	$-1,213604 - 0,322864i$
3	$-1,378383 + 0,540232i$	11	$-0,991293 - 1,391733i$
4	$0,080000 + 1,160000i$	12	$0,080000 - 1,160000i$
5	$-0,991293 + 1,391733i$	13	$-1,378383 - 0,540232i$
6	$-1,213604 + 0,322864i$	14	$-2,486396 - 5,022864i$
7	$-0,800149 + 0,042971i$	15	$5,529825 - 9,351469i$

Результат обратного преобразования совпадает с исходным сигналом с точностью до машинной погрешности.

Таблица 3: Проверка обратного ДПФ.

j	y_j	$\hat{y}_j$	$ y_j - \hat{y}_j $
0	$-0,780000$	$-0,780000$	$4,56 \cdot 10^{-15}$
1	$-1,220000$	$-1,220000$	$3,97 \cdot 10^{-15}$
2	$-1,340000$	$-1,340000$	$4,70 \cdot 10^{-15}$
3	$-1,390000$	$-1,390000$	$2,20 \cdot 10^{-15}$
4	$-1,420000$	$-1,420000$	$3,33 \cdot 10^{-15}$
5	$-1,430000$	$-1,430000$	$3,71 \cdot 10^{-15}$
6	$-1,420000$	$-1,420000$	$2,18 \cdot 10^{-15}$
7	$-1,410000$	$-1,410000$	$1,82 \cdot 10^{-15}$
8	$-1,370000$	$-1,370000$	$1,11 \cdot 10^{-15}$
9	$-1,300000$	$-1,300000$	$4,60 \cdot 10^{-15}$
10	$-1,100000$	$-1,100000$	$3,30 \cdot 10^{-15}$
11	$-0,100000$	$-0,100000$	$2,37 \cdot 10^{-15}$
12	$1,120000$	$1,120000$	$3,34 \cdot 10^{-15}$
13	$1,250000$	$1,250000$	$2,26 \cdot 10^{-15}$
14	$1,330000$	$1,330000$	$7,34 \cdot 10^{-15}$
15	$1,360000$	$1,360000$	$1,57 \cdot 10^{-15}$

4.2 Быстрое преобразование Фурье

После разделения исходного сигнала на отсчёты с чётными и нечётными индексами получены следующие вспомогательные векторы амплитуд:

Таблица 4: Векторы амплитуд $F^0(k)$ и $F^1(k)$.

k	$F^0(k)$	$F^1(k)$
0	-4,980000	-4,240000
1	$2,364838 + 4,654249i$	$1,126518 + 5,550854i$
2	$-1,850000 + 2,350000i$	$-2,340000 + 1,440000i$
3	$-1,184838 - 0,425751i$	$-0,966518 + 0,190854i$
4	0,080000	-1,160000
5	$-1,184838 + 0,425751i$	$-0,966518 - 0,190854i$
6	$-1,850000 - 2,350000i$	$-2,340000 - 1,440000i$
7	$2,364838 - 4,654249i$	$1,126518 - 5,550854i$

По формулам БПФ получен тот же вектор амплитуд $F(k)$, что и при прямом ДПФ (таблица 2). Максимальное отличие между прямым ДПФ и БПФ составило $6,58 \cdot 10^{-14}$.

Векторы обратных преобразований по чётным и нечётным отсчётам:

Таблица 5: Векторы $\hat{y}^0(j)$ и $\hat{y}^1(j)$.

j	$\hat{y}^0(j)$	$\hat{y}^1(j)$
0	-0,780000	-1,220000
1	-1,340000	-1,390000
2	-1,420000	-1,430000
3	-1,420000	-1,410000
4	-1,370000	-1,300000
5	-1,100000	-0,100000
6	1,120000	1,250000
7	1,330000	1,360000

После объединения этих подвекторов восстановлен исходный сигнал. Максимальная погрешность восстановления составила $2,50 \cdot 10^{-15}$.

Как видно из таблиц, обратные преобразования возвращают исходные отсчёты, а отклонения имеют порядок машинной точности.

5 Приложение

Listing 1: Файл main.py

```
1 import cmath
2 import math
3 import re
4 from pathlib import Path
5
6
7 ROOT = Path(__file__).resolve().parents[1]
8 VARIANT_FILE = ROOT / "task" / "var.md"
9 EPS = 1e-10
10
11
12 def parse_variant(path):
13     text = path.read_text(encoding="utf-8")
14
15     variant_match = re.search(r"variant:\s*(\d+)", text)
16     values_match = re.search(r"values:\s*(.+)", text)
17     if not variant_match or not values_match:
18         raise ValueError("File var.md must contain 'variant:' and
19                             'values:' lines.")
20
21     variant = int(variant_match.group(1))
22     values = [float(item.replace(",", ".")) for item in
23               values_match.group(1).split()]
24     return variant, values
25
26
27 def dft(values):
28     n = len(values)
29     result = []
30     for k in range(n):
31         total = 0j
32         for j, value in enumerate(values):
33             total += value * cmath.exp(-2j * math.pi * k * j / n)
34         result.append(total)
35     return result
36
37
38 def inverse_dft(amplitudes):
```

```

37     n = len(amplitudes)
38     result = []
39     for j in range(n):
40         total = 0j
41         for k, value in enumerate(amplitudes):
42             total += value * cmath.exp(2j * math.pi * k * j / n)
43         result.append(total / n)
44     return result
45
46
47 def fft_decimation_by_time(values):
48     n = len(values)
49     if n % 2 != 0:
50         raise ValueError("The number of samples must be even for
radix-2 decimation.")
51
52     even_values = values[0::2]
53     odd_values = values[1::2]
54     even_amplitudes = dft(even_values)
55     odd_amplitudes = dft(odd_values)
56
57     amplitudes = [0j] * n
58     for k in range(n // 2):
59         multiplier = cmath.exp(-2j * math.pi * k / n)
60         amplitudes[k] = even_amplitudes[k] + multiplier *
odd_amplitudes[k]
61         amplitudes[k + n // 2] = even_amplitudes[k] - multiplier
* odd_amplitudes[k]
62
63     return even_values, odd_values, even_amplitudes,
odd_amplitudes, amplitudes
64
65
66 def inverse_fft_decimation_by_time(even_amplitudes,
odd_amplitudes):
67     even_values = inverse_dft(even_amplitudes)
68     odd_values = inverse_dft(odd_amplitudes)
69
70     values = []
71     for even_value, odd_value in zip(even_values, odd_values):
72         values.append(even_value)

```

```

73         values.append(odd_value)
74
75     return even_values, odd_values, values
76
77
78 def clean_number(value, digits=6):
79     if abs(value) < EPS:
80         value = 0.0
81     return f"{value:.{digits}f}"
82
83
84 def format_complex(value, digits=6):
85     real = 0.0 if abs(value.real) < EPS else value.real
86     imag = 0.0 if abs(value.imag) < EPS else value.imag
87
88     if imag == 0.0:
89         return clean_number(real, digits)
90     if real == 0.0:
91         return f"{clean_number(imag, digits)}i"
92
93     sign = "+" if imag >= 0 else "-"
94     return f"{clean_number(real, digits)} {sign} {clean_number(
95         abs(imag), digits)}i"
96
97
98 def print_real_vector(title, values):
99     print(title)
100     print(f"{'j':>3} {'value':>18}")
101     for j, value in enumerate(values):
102         number = value.real if isinstance(value, complex) else
103         value
104         print(f"{'j':>3} {clean_number(number):>18}")
105     print()
106
107 def print_complex_vector(title, values, index_name="k"):
108     print(title)
109     print(f"{index_name:>3} {'value':>28}")
110     for index, value in enumerate(values):
111         print(f"{index:>3} {format_complex(value):>28}")
112     print()

```

```

112
113
114 def print_verification(title , original , restored , index_name="j",
    original_name="original", restored_name="restored"):
115     print(title)
116     print(f"{index_name:>3} {original_name:>14} {restored_name
    :>28} {'abs error':>14}")
117     max_error = 0.0
118     for j, (source , value) in enumerate(zip(original , restored)):
119         error = abs(source - value)
120         max_error = max(max_error , error)
121         print(
122             f"{j:>3} "
123             f"{format_complex(source):>14} "
124             f"{format_complex(value):>28} "
125             f"{error:>14.3e}"
126         )
127     print(f"max error = {max_error:.3e}")
128     print()
129
130
131 def main():
132     variant , values = parse_variant(VARIANT_FILE)
133
134     print("=" * 78)
135     print("LAB 5: DISCRETE FOURIER TRANSFORM")
136     print("=" * 78)
137     print(f"Variant: {variant}")
138     print(f"Number of samples: n = {len(values)}")
139     print()
140
141     print_real_vector("Initial signal y(j):", values)
142
143     print("-" * 78)
144     print("TASK 1. Direct DFT and inverse DFT")
145     print("-" * 78)
146     amplitudes = dft(values)
147     restored = inverse_dft(amplitudes)
148
149     print_complex_vector("DFT amplitudes F(k):", amplitudes)

```

```

150     print_complex_vector("Inverse DFT result y(j):", restored ,
index_name="j")
151     print_verification("DFT reconstruction check:", values ,
restored)

152
153     print("--" * 78)
154     print("TASK 2. Fast Fourier transform by time decimation")
155     print("--" * 78)
156     (
157         even_values ,
158         odd_values ,
159         even_amplitudes ,
160         odd_amplitudes ,
161         fft_amplitudes ,
162     ) = fft_decimation_by_time(values)

163
164     print_real_vector("Even-index samples y^0(j) = y(2j):",
even_values)
165     print_real_vector("Odd-index samples y^1(j) = y(2j + 1):",
odd_values)
166     print_complex_vector("DFT by even samples F^0(k):",
even_amplitudes)
167     print_complex_vector("DFT by odd samples F^1(k):",
odd_amplitudes)
168     print_complex_vector("FFT amplitudes F(k):", fft_amplitudes)

169
170     even_restored , odd_restored , fft_restored =
inverse_fft_decimation_by_time(
171         even_amplitudes ,
172         odd_amplitudes ,
173     )

174
175     print_complex_vector("Inverse transform by even samples y^0(j
):", even_restored , index_name="j")
176     print_complex_vector("Inverse transform by odd samples y^1(j
):", odd_restored , index_name="j")
177     print_complex_vector("Restored signal from inverse FFT blocks
y(j):", fft_restored , index_name="j")

178
179     print_verification(
180         "FFT amplitudes compared with direct DFT:",

```

```

181         amplitudes ,
182         fft_amplitudes ,
183         index_name="k",
184         original_name="direct DFT",
185         restored_name="FFT",
186     )
187     print_verification("FFT reconstruction check:", values ,
188                       fft_restored)
189
189     print("==" * 78)
190     print("DONE")
191     print("==" * 78)
192
193
194 if __name__ == "__main__":
195     main()

```